

Лабораторная работа №1

Начало работы с MySQL. MySQL Workbench

Кротов Иван Сергеевич, группа ИВТ-2

Цель работы: познакомиться с инструментом MySQL Workbench, который позволяет работать с базой данных типа MySQL, предлагаемой компанией Oracle.

Задание 1

Раздел «Management» («Управление»)

Этот раздел содержит инструменты для администрирования сервера, управления пользователями и их привилегиями, а также для настройки резервного копирования.

1. Раздел «Server Status» (Статус сервера)

В разделе отображается общая информация о сервере и подключении к нему. Информация логически сгруппирована. Можно выделить следующие группы:

- Общая информация (например, название хоста, номер порта, версия БД)
- Настройки сервера (например, включен ли брандмауэр, используется ли SSL)
- Каталоги сервера
- Сводка по используемым ресурсам компьютера (ОЗУ, процессор и т. д.)
- Настройки соединения SSL (если SSL включена)

2. Раздел «Client Connections» (Клиентские подключения)

Предназначен для мониторинга и управления активными подключениями к серверу MySQL. Здесь можно:

- Просматривать список подключений (отображается информация о каждом подключении: идентификатор процесса, пользователь, хост, текущая выполняемая команда, время бездействия и используемая схема)
- Завершать проблемные подключения (администратор может принудительно завершить конкретное соединение, например, если оно блокирует работу других пользователей или выполняет слишком долгий запрос)

3. Раздел «Status and System Variables» (Переменные состояния и системы)

Предоставляет полный список всех серверных переменных (статуса и системных) для активного подключения. Переменные можно фильтровать, группировать по категориям, а также просматривать их описание и текущие значения.

4. Раздел «Users and Privileges» (Пользователи и привилегии)

Служит для управления учетными записями пользователей MySQL. Здесь можно:

- Управлять учетными записями (добавлять, удалять и блокировать пользователей, задавать для них пароли и хосты для подключения)
- Назначать привилегии (выдавать права на глобальном уровне, уровне схемы или конкретных объектов через удобные вкладки)

5. Разделы «Data Export» и «Data Import/Restore»

Предназначены для создания логических резервных копий баз данных и их последующего восстановления, что является ключевой задачей администрирования.

Раздел «Instance» («Экземпляр БД»)

Этот раздел фокусируется на прямом управлении и конфигурировании работающего экземпляра сервера MySQL.

1. Раздел «Startup / Shutdown» (Запуск / Остановка)

Предоставляет интерфейс для управления службой сервера MySQL. Здесь можно просматривать журнал запуска (Startup Message Log), а также выполнять ключевые действия по контролю за сервисом: остановка (Stop) или запуск (Start) экземпляра базы данных.

2. Раздел «Server Logs» (Журналы сервера)

Предназначен для просмотра файлов журналов сервера MySQL. Информация разделена по вкладкам:

- Error Log (Журнал ошибок) - содержит записи о критических ошибках, предупреждениях и событиях запуска/остановки сервера

- Slow Log (Журнал медленных запросов) – отображает запросы (SQL-операторы), время выполнения которых превысило пороговое значение, помогая выявить неэффективные запросы для оптимизации

3. Раздел «Options File» (Файл опций)

Инструмент для редактирования конфигурационного файла MySQL (my.ini или my.cnf) через графический интерфейс. Позволяет изменять параметры сервера (например, пути к данным, настройки логирования, параметры InnoDB) на удобных вкладках, таких как General, после чего изменения можно применить (Apply).

Раздел «Performance» («Производительность»)

1. Раздел «Dashboard» (Инструментальная панель)

Обеспечивает визуальный мониторинг ключевых метрик сервера в реальном времени в виде графиков, позволяя быстро оценить общее состояние системы. Можно выделить следующие области:

- Сетевой статус (отображение входящего и исходящего сетевого трафика, а также количества активных подключений клиентов)
- Статус MySQL (графики, показывающие интенсивность выполнения различных типов SQL-операторов и эффективность работы кэша)
- Статус InnoDB (детальная информация о работе движка InnoDB: операции чтения/записи, использование буферного пула и другие метрики)

2. Раздел «Performance Reports» (Отчеты о производительности)

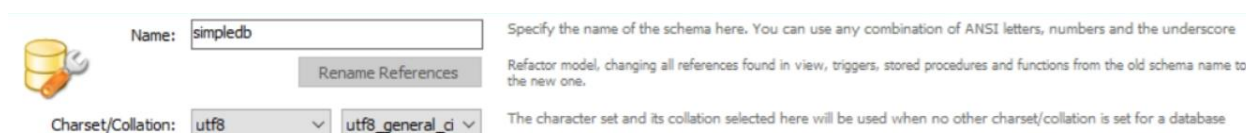
Содержит библиотеку структурированных отчетов для углубленного анализа работы сервера за период. Отчеты используют данные из Performance Schema и системной схемы SYS. С их помощью можно, например, найти файлы с максимальной нагрузкой ввода-вывода (Top File I/O Activity) или выявить самые затратные SQL-запросы (Statement Analysis) .

3. Раздел « Performance Schema Setup» (Настройка Performance Schema)

Предоставляет интерфейс для конфигурации Performance Schema (механизма мониторинга сервера MySQL на низком уровне). Вкладка Easy Setup позволяет быстро включать или отключать наборы инструментов мониторинга, что упрощает настройку для большинства пользователей.

Задания 2 и 3

Необходимо создать и настроить новую базу данных под названием `simpledb`. Для создания новой базы данных в верхнем меню нажимаем на кнопку «Создание новой базы данных в рамках данного подключения к серверу», которая находится в панели главного окна. Далее в настройках указываем имя `simpledb`, кодировку `utf-8` и сопоставление `utf8_general_ci`, после чего нажимаем `Apply`, видим SQL-запрос, равносильный выбранным настройкам, снова нажимаем `Apply` и затем `Finish`.



Name:

Charset/Collation:

Rename References

Specify the name of the schema here. You can use any combination of ANSI letters, numbers and the underscore

Refactor model, changing all references found in view, triggers, stored procedures and functions from the old schema name to the new one.

The character set and its collation selected here will be used when no other charset/collation is set for a database

Рисунок 1

Готово, новая база данных создана. Выставим эту базу данных как БД по умолчанию. Далее, создадим несколько таблиц в этой базе данных. Для этого раскроем с помощью треугольника содержимое этой БД. В контекстном меню по пункту `Tables`, выберем пункт `Create Table`. Создадим таблицу `users` со следующими полями и их характеристиками:

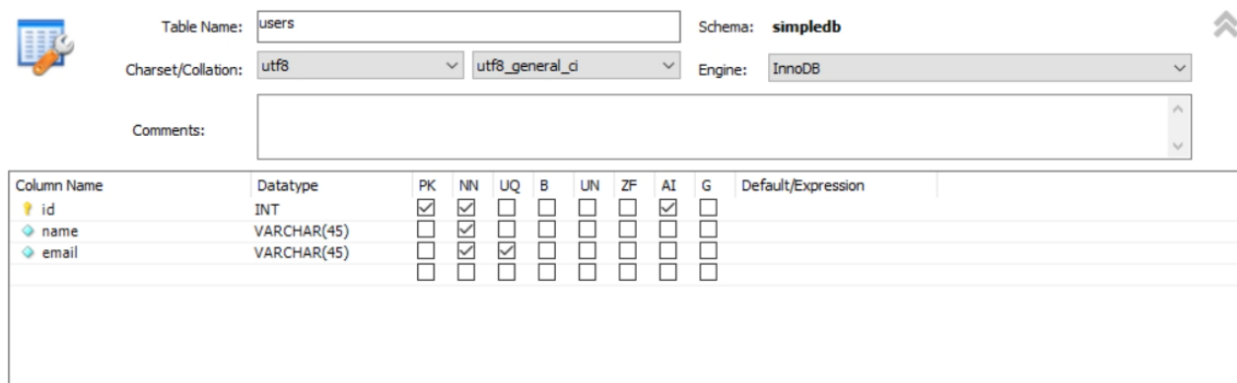


Table Name: Schema: **simpledb**

Charset/Collation: Engine:

Comments:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
email	VARCHAR(45)	☐	☒	☒	☐	☐	☐	☐	☐	

Рисунок 2

Теперь нажмём `Apply`, после чего получим следующий SQL-запрос для создания таблицы с указанными ранее настройками (этот код – ответ на задание 3):

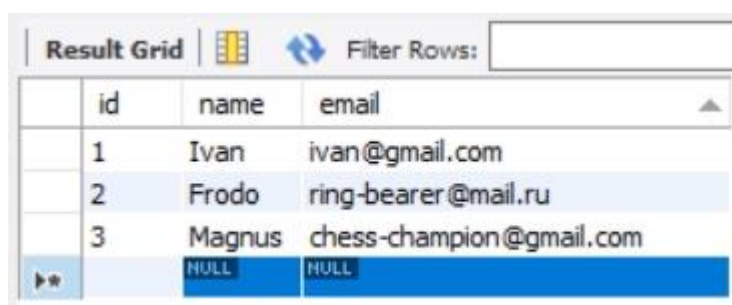
```
1 CREATE TABLE `simpledb`.`users` (  
2   `id` INT NOT NULL AUTO_INCREMENT,  
3   `name` VARCHAR(45) NOT NULL,  
4   `email` VARCHAR(45) NOT NULL,  
5   PRIMARY KEY (`id`),  
6   UNIQUE INDEX `email_UNIQUE` (`email` ASC) VISIBLE)  
7 ENGINE = InnoDB  
8 DEFAULT CHARACTER SET = utf8;  
9
```

Рисунок 3

Задание 4

Теперь необходимо добавить несколько примеров записей в созданную таблицу. Для этого кликаем на название таблицы левой кнопкой мыши и выбираем там пиктограмму, где изображена таблица с молнией.

Далее в нижней части экрана (где таблица) кликая по столбцам придумаем и введём 3 новые строки, которые будут храниться в таблице. Значения для первого столбца (id) не вводим, потому что он будет заполняться автоматически. После ввода всех строк нажимаем Apply и сохраняем изменения в таблице. На рисунке 4 представлен результат работы, а на рисунке 5 – равносильный SQL-запрос.



	id	name	email
	1	Ivan	ivan@gmail.com
	2	Frodo	ring-bearer@mail.ru
	3	Magnus	chess-champion@gmail.com
▶*		NULL	NULL

Рисунок 4

```
1  INSERT INTO `simplifiedb`.`users` (`id`, `name`, `email`) VALUES ('1', 'Ivan', 'ivan@gmail.com');
2  INSERT INTO `simplifiedb`.`users` (`id`, `name`, `email`) VALUES ('2', 'Frodo', 'ring-bearer@mail.ru');
3  INSERT INTO `simplifiedb`.`users` (`id`, `name`, `email`) VALUES ('3', 'Magnus', 'chess-champion@gmail.com');
4
```

Рисунок 5

Далее нужно внести изменения в какие-нибудь поля в таблице выше. Я решил изменить почту у пользователя Ivan вручную. Аналогичный этому действию SQL-запрос представлен на рисунке 6. Это UPDATE-запрос, который изменяет значение в указанном поле email с помощью SET там, где (WHERE) id равен определённому значению, в данном случае 1.

```
1  UPDATE `simplifiedb`.`users` SET `email` = 'ivan-krotov@gmail.com' WHERE (`id` = '1');
2
```

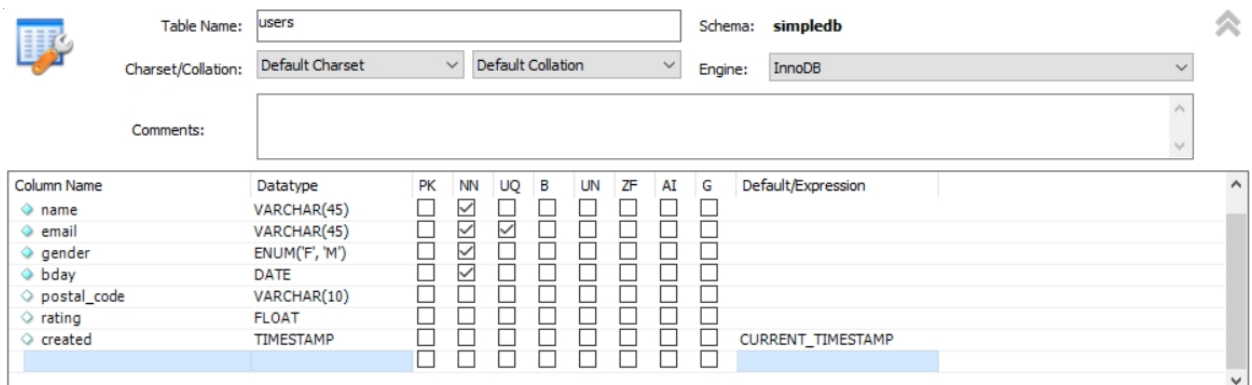
Рисунок 6

Задание 5

Теперь дополним таблицу users так, чтобы получилась таблица с такими полями и параметрами:

1. id, int pk, not null
2. name varchar(45)
3. email varchar(45)
4. gender ENUM('M', 'F')
5. bday Date
6. postal_code varchar(10)
7. rating float
8. created TIMESTAMP CURRENT_TIMESTAMP()

Для этого нажмём на иконку с гаечным ключом напротив редактируемой таблицы и внесём необходимые изменения. Результат (перед применением изменений) продемонстрирован на рисунке 7, а аналогичный SQL-запрос представлен на рисунке 8.



Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
name	VARCHAR(45)	☒	☒	☐	☐	☐	☐	☐	☐	
email	VARCHAR(45)	☐	☒	☒	☐	☐	☐	☐	☐	
gender	ENUM('F', 'M')	☐	☒	☐	☐	☐	☐	☐	☐	
bday	DATE	☐	☒	☐	☐	☐	☐	☐	☐	
postal_code	VARCHAR(10)	☐	☐	☐	☐	☐	☐	☐	☐	
rating	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	
created	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP

Рисунок 7

```
1 ALTER TABLE `simplifiedb`.`users`  
2 ADD COLUMN `gender` ENUM('F', 'M') NOT NULL AFTER `email`,  
3 ADD COLUMN `bday` DATE NOT NULL AFTER `gender`,  
4 ADD COLUMN `postal_code` VARCHAR(10) NULL AFTER `bday`,  
5 ADD COLUMN `rating` FLOAT NULL AFTER `postal_code`,  
6 ADD COLUMN `created` TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP AFTER `rating`;  
7
```

Рисунок 8

TIMESTAMP DEFAULT CURRENT_TIMESTAMP() - Это значит, что поле автоматически заполняется текущей датой и временем в момент добавления новой записи в таблицу.

Не вся информация обязательная. Не все пользователи захотят делиться необязательной информацией. Такие поля могут быть NULL. Обязательными для заполнения (помимо name и email был принято решение сделать только поля gender и bday. Остальные могут иметь значение NULL.

Задание 6

Теперь необходимо дополнить таблицу с помощью выполнения SQL-запросов, приведённых ниже (внесение данных вручную было выполнено в прошлых заданиях):

```
INSERT INTO `simplifiedb`.`users` (`name`, `email`, `postal_code`, `gender`, `bday`, `rating`)
VALUES ('Ekaterina', 'ekaterina.petrova@outlook.com', '145789', 'f', '2000-02-11', '1.123');
```

```
INSERT INTO `simplifiedb`.`users` (`name`, `email`, `postal_code`, `gender`, `bday`, `rating`)
VALUES ('Paul', 'paul@superpochta.ru', '123789', 'm', '1998-08-12', '1');
```

Для этого используем кнопку (1) в главном окне программы. В открывшемся окне вписываем эти SQL-запросы и применяем нажатием на значок молнии (Рисунок 10). После этого таблица users обновится, и в ней появятся новые данные (Рисунок 11).



Рисунок 10

Result Grid								
Filter Rows:								
Edit:								
Export/Import:								
Wrap Cell Content:								
	id	name	email	gender	bday	postal_code	rating	created
	1	Ivan	ivan-krotov@gmail.com	M	2005-07-07	NULL	NULL	2026-03-19 22:21:17
	2	Frodo	ring-bearer@mail.ru	M	1981-01-28	NULL	NULL	2026-03-19 22:21:17
	3	Magnus	chess-champion@gmail.com	M	1990-11-30	NULL	NULL	2026-03-19 22:21:17
	4	Ekaterina	ekaterina.petrova@outlook.com	F	2000-02-11	145789	1.123	2026-03-19 22:41:01
	5	Paul	paul@superpochta.ru	M	1998-08-12	123789	1	2026-03-19 22:41:01
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Рисунок 11

Задание 7

Далее нужно с помощью кнопки Export recordset to external file получить файл с SQL-запросами (он будет экспортирован в формате .sql) и сохранить его (Рисунок 12). В сохранённом файле находятся SQL-запросы, представленные на рисунке 13 (полностью поместить на одном скриншоте их не удалось, поэтому в конце не видно часть данных).

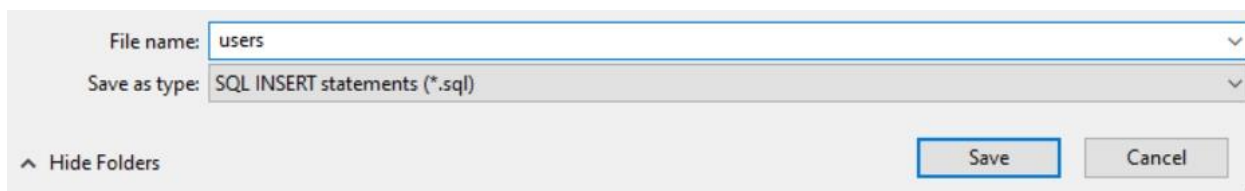


Рисунок 12

```
1  /*
2  -- Query: SELECT * FROM simbledb.users
3  LIMIT 0, 1000
4
5  -- Date: 2026-03-20 08:40
6  */
7  INSERT INTO `(`id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES (1,'Ivan','ivan-krotov@gmail.com','M','2005-07-07',NULL,NULL,'2026-03-19 22:
8  INSERT INTO `(`id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES (2,'Frodo','ring-bearer@mail.ru','M','1901-01-28',NULL,NULL,'2026-03-19 22:2
9  INSERT INTO `(`id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES (3,'Magnus','chess-champion@gmail.com','M','1990-11-30',NULL,NULL,'2026-03-1
10 INSERT INTO `(`id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES (4,'Ekaterina','ekaterina.petrova@outlook.com','F','2000-02-11','145789',1.1
11 INSERT INTO `(`id`,`name`,`email`,`gender`,`bday`,`postal_code`,`rating`,`created`) VALUES (5,'Paul','paul@superpochta.ru','M','1998-08-12','123789',1,'2026-03-19 22:4
12
```

Рисунок 13

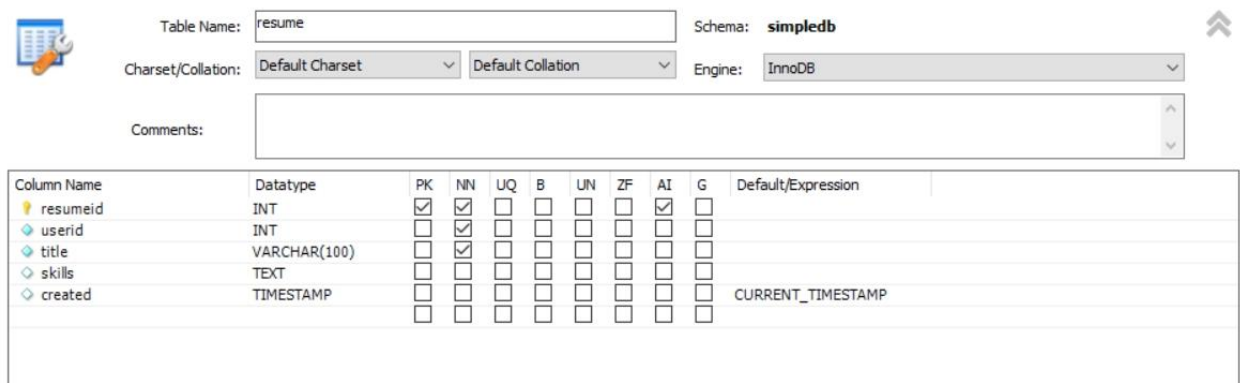
Анализируя содержимое полученного файла, видим, что в комментарии указано, что это выборка данных из таблицы users из базы данных simbledb, при этом размер файла ограничен 1000 строк, а дата его создания – 20 марта 2026 года.

Сам файл содержит INSERT-запросы, которые были сгенерированы при экспорте данных. Каждый INSERT добавляет одну строку в таблицу users с конкретными значениями: id, имя, email, пол, дата рождения, почтовый индекс, рейтинг и дата создания записи.

Задание 8

Теперь создадим ещё одну таблицу с названием `resume` со следующей структурой:

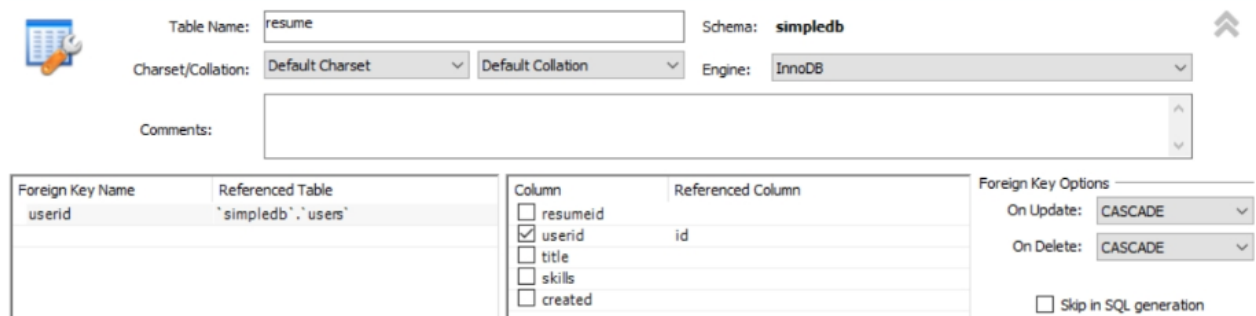
- `resumeid`, INT, PK, NN, AI
- `userid`, INT, NN
- `title`, VARCHAR(100), NN
- `skills`, TEXT
- `created`, TIMESTAMP, Default / Expression: `CURRENT_TIMESTAMP()`



Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
resumeid	INT	☒	☒	☐	☐	☐	☐	☒	☐	
userid	INT	☐	☒	☐	☐	☐	☐	☐	☐	
title	VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐	
skills	TEXT	☐	☐	☐	☐	☐	☐	☐	☐	
created	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP

Рисунок 14

При этом при конструировании таблицы внизу во вкладке **Foreign keys** необходимо определить так называемый внешний ключ, который будет определять связь между текущей таблицей `resume` и уже созданной таблицей `users`. В таблицу слева в столбец **Foreign key** введём `userid` и определим, где они будут находиться, а именно в `simpledb.users`. В таблице **Foreign key details** поставим галочку рядом с полем `userid`, чтобы оно было выделено, и определим столбец-источник для значений, то есть `id`. Последний шаг: определим действия при операциях **ON UPDATE** и **ON DELETE**, а именно для обоих выберем вариант **CASCADE**. Нажимаем **Apply** и получаем SQL-запрос.



Foreign Key Name	Referenced Table	Column	Referenced Column
userid	simpledb	userid	id

Foreign Key Options

On Update: **CASCADE**

On Delete: **CASCADE**

☐ Skip in SQL generation

Рисунок 15

```

1  CREATE TABLE `simplifiedb`.`resume` (
2      `resumeid` INT NOT NULL AUTO_INCREMENT,
3      `userid` INT NOT NULL,
4      `title` VARCHAR(100) NOT NULL,
5      `skills` TEXT NULL,
6      `created` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
7      PRIMARY KEY (`resumeid`),
8      INDEX `fk_resume_users_idx` (`userid` ASC) VISIBLE,
9      CONSTRAINT `fk_resume_users`
10     FOREIGN KEY (`userid`)
11     REFERENCES `simplifiedb`.`users` (`id`)
12     ON DELETE CASCADE
13     ON UPDATE CASCADE
14 ) ENGINE = InnoDB DEFAULT CHARACTER SET = utf8 COLLATE = utf8_general_ci;

```

Рисунок 16

Анализируя SQL-запрос, видим, что он создает таблицу resume в базе данных simplifiedb. Таблица включает пять столбцов: resumeid (первичный ключ с автозаполнением), userid (идентификатор пользователя), title («название» резюме), skills (навыки), created (дата создания, по умолчанию текущее время). Внешний ключ связывает поле userid с id из таблицы users. Настроены каскадные действия: при обновлении или удалении пользователя изменения автоматически применяются к связанным резюме (ON UPDATE CASCADE и ON DELETE CASCADE). Для ускорения работы создан индекс. Таблица использует движок InnoDB и кодировку utf8.

Задание 9

Далее наполним таблицу resume данными так, чтобы в ней была информация хотя бы о нескольких резюме, связанных с уже существующими пользователями из таблицы users. Результат представлен на рисунке 17.

resumeid	userid	title	skills	created
1	1	Student	Procrastination	2026-03-20 15:42:24
2	2	Ring bearer	Endurance, stealth	2026-03-20 15:42:24
3	3	Chess player	Strategy, tactics	2026-03-20 15:42:24
▶*	NULL	NULL	NULL	NULL

Рисунок 17

Сколько резюме может быть у одного пользователя? Минимум: 0 резюме. Таблица resume не требует, чтобы у каждого пользователя обязательно было резюме. Поэтому пользователь мог зарегистрироваться, но ещё не создать резюме. Максимум же у пользователя может быть неограниченное количество резюме. Точнее, ограниченное, но без дополнительных ограничений это число будет очень большим.

Затем с помощью кнопки «Export recordset to external file» сохраним файл с SQL-запросами добавления данных в таблицу. Полученные SQL-запросы представлены на рисунке 18.

```
1  INSERT INTO `simplifiedb`.`resume` (`resumeid`, `userid`, `title`, `skills`) VALUES ('1', '1', 'Student', 'Procrastination');
2  INSERT INTO `simplifiedb`.`resume` (`resumeid`, `userid`, `title`, `skills`) VALUES ('2', '2', 'Ring bearer', 'Endurance, stealth');
3  INSERT INTO `simplifiedb`.`resume` (`resumeid`, `userid`, `title`, `skills`) VALUES ('3', '3', 'Chess player', 'Strategy, tactics');
4
```

Рисунок 18

При попытке добавить резюме с несуществующим userid возникла ошибка внешнего ключа. Поэтому если такого пользователя нет, то запись не создастся.

Задание 10

Теперь необходимо удалить какого-нибудь пользователя из таблицы users, что у него есть резюме в таблице resume. Для этого был выбран пользователь Frodo. Переходим в таблицу users, кликаем правой кнопкой мыши по строчке с пользователем Frodo, выбираем Delete Row и затем Apply. Получим SQL-запрос на удаление:

```
1  DELETE FROM `simplifiedb`.`users` WHERE (`id` = '2');
2
```

Рисунок 19

Что произойдёт со связанным резюме из таблицы resume? Если мы перейдём в таблицу resume, то увидим, что связанное с удалённым пользователем резюме исчезло. Это произошло, потому что при создании было выбрано ON DELETE CASCADE. Это значит, что при удалении родительских данных (в данном случае пользователя) исчезают и связанные с ними (резюме). А что же произойдёт, если изменить ID пользователя, у которого есть резюме? А всё то же самое: при создании была выбрана опция ON UPDATE CASCADE, поэтому при изменении ID пользователя в таблице users, оно изменится и у его резюме в таблице resume.